Manual de creación de la máquina virtual

Este manual describe, paso a paso, cómo se creó desde cero una máquina virtual con **Debian 13** sobre **KVM** y cómo se desplegó en ella **WordPress** servido por **Nginx**, **PHP-FPM** y **MariaDB**. La máquina se construyó a partir de la imagen *cloud* oficial de Debian configurada con **cloud-init**, el mismo mecanismo que usan las instancias de AWS, y la aplicación se ejecuta en contenedores gestionados con **Podman**.

1. Selección del entorno de virtualización

Se eligió **KVM** (*Kernel-based Virtual Machine*) con **QEMU** y **libvirt**, administrado desde la línea de comandos con **virt-install** y **virsh**. La actividad recomienda VirtualBox por su sencillez, pero KVM se consideró más adecuado por los siguientes motivos:

- **Hipervisor integrado en el núcleo de Linux:** aprovecha directamente las extensiones de virtualización del procesador (Intel VT-x o AMD-V), con un rendimiento cercano al nativo; además, con `--cpu host-passthrough` la VM ve las mismas instrucciones que la CPU física.
- **Dispositivos paravirtualizados (virtio)** para disco, red y vídeo, y **software libre** incluido en los repositorios, sin licencias.
- **Relación directa con la nube:** el hipervisor Nitro de las instancias EC2 de AWS está basado en la tecnología de KVM (Amazon Web Services, s. f.), y las imágenes *cloud* con cloud-init funcionan igual en ambos entornos.

2. Creación de la máquina virtual desde cero

En lugar del instalador interactivo se utilizó la imagen **debian-13-genericcloud-amd64.qcow2**, publicada por Debian para entornos virtualizados. Su configuración inicial (usuario, llave SSH, paquetes y nombre) se define con **cloud-init**, de modo que la máquina queda lista en el primer arranque y el proceso es reproducible.

Tabla 1. Características de la máquina virtual.

Parámetro	Valor
Nombre / hostname	debian13-actividad2
Sistema operativo	Debian 13 «trixie», 64 bits (imagen genericcloud)
CPU y memoria	2 vCPU (host-passthrough) y 4 GB de RAM

Parámetro	Valor
Disco	100 GB, formato qcow2 con bus virtio (asignación dinámica)
Red	Red NAT «default» de libvirt, tarjeta virtio (IP 192.168.124.87)
Acceso	Usuario miguelguzman, solo con llave SSH ed25519; root y contraseñas deshabilitados

Pasos realizados:

1. Copiar la imagen base a un disco propio para la VM y ampliarlo a 100 GB; cloud-init extiende la partición automáticamente en el primer arranque:

```
cd /var/lib/libvirt/images/
sudo cp debian-13-genericcloud-amd64.qcow2 \
        debian-13-actividad2-maestria.qcow2
sudo qemu-img resize debian-13-actividad2-maestria.qcow2 100G
sudo qemu-img info debian-13-actividad2-maestria.qcow2
```

2. Crear el archivo `cloud_debian_maestria.yaml` con la configuración inicial (extracto):

```
#cloud-config
hostname: debian13-actividad2
users:
  - name: miguelguzman
    groups: [sudo]
    sudo: ALL=(ALL) NOPASSWD:ALL
    lock_passwd: true
    ssh_authorized_keys: [ssh-ed25519 AAAA... miguelguzman@anfitrion]
disable_root: true
ssh_pwauth: false
package_update: true
packages: [qemu-guest-agent, openssh-server, curl, vim, git, htop, jq]
runcmd:
  - systemctl enable --now qemu-guest-agent
```

3. Crear e importar la máquina con virt-install, pasando el archivo de cloud-init:

```
sudo virt-install --name debian13-actividad2 \
  --memory 4096 --vcpus 2 --cpu host-passthrough \
  --os-variant debian13 \
  --disk path=$(pwd)/debian-13-actividad2-maestria.qcow2,\
        format=qcow2,bus=virtio \
  --import --network network=default,model=virtio \
  --graphics spice --video virtio \
  --cloud-init user-data=$(pwd)/cloud_debian_maestria.yaml
```

4. Comprobar la máquina, obtener su IP y conectarse por SSH:

```
sudo virsh list --all --name
sudo virsh dominfo debian13-actividad2
sudo virsh domifaddr debian13-actividad2      # 192.168.124.87
ssh miguelguzman@192.168.124.87
```

5. Actualizar el sistema con `sudo apt update && sudo apt full-upgrade -y`.

6. **Generalizar la imagen base** para poder reutilizar el disco como plantilla: `sudo cloud-init clean`, vaciar `/etc/machine-id` con `sudo truncate -s 0 /etc/machine-id` y eliminar `/var/lib/dbus/machine-id`. Así cada copia de la máquina genera identificadores propios en su siguiente arranque, el mismo principio que se aplica al crear una imagen (AMI) en AWS.

3. Instalación de la aplicación web

Se seleccionó **WordPress**, una de las aplicaciones propuestas. En lugar de Apache se utilizó **Nginx** con **PHP-FPM**, una combinación muy extendida en producción, y **MariaDB** como base de datos. Cada componente se ejecuta en su propio contenedor con **Podman en modo sin privilegios** (*rootless*), bajo un **usuario de sistema dedicado** llamado `podman`, separado del usuario administrador.

Tabla 2. Contenedores de la aplicación.

Contenedor	Imagen	Función
wp-nginx	localhost/wp-nginx:1.0 (basada en nginx:1.28-alpine)	Recibe las peticiones HTTP en el puerto 8080, sirve los archivos estáticos y envía los .php a PHP-FPM.
wp-app	localhost/wp-app:1.0 (basada en wordpress:6.8-php8.3-fpm)	Ejecuta WordPress con PHP-FPM; límites de subida ampliados a 64 MB.
wp-db	mariadb:11.4	Base de datos. Solo accesible desde la red interna de los contenedores.

Pasos realizados:

1. Instalar las utilidades base y Podman: `sudo apt install -y podman pipx unzip ca-certificates gnupg` y comprobar con `podman version`.
2. Crear el usuario de sistema que ejecutará los contenedores e instalar en él la versión más reciente de podman-compose:

```
sudo useradd --system --create-home --home-dir /srv/podman \
        --shell /bin/bash podman
sudo -iu podman
pipx install podman-compose && pipx ensurepath && source ~/.bashrc
podman-compose --version       # 1.6.0
exit
```

3. Asignar rangos de UID/GID subordinados al usuario, necesarios para los *user namespaces* de Podman sin privilegios, y habilitar el arranque de sus contenedores sin sesión iniciada:

```
sudo usermod --add-subuids 100000-165535 --add-subgids 100000-165535 podman
sudo loginctl enable-linger podman            # loginctl show-user podman ->
Linger=yes
sudo systemctl --machine=podman@.host --user enable --now podman-restart.service
sudo -iu podman
podman system migrate
podman unshare cat /proc/self/uid_map         # 0  100000  65536
```

4. Como usuario `podman`, crear la estructura del proyecto en `~/unir/wordpress` (subcarpetas `nginx` y `wordpress`), copiar `compose.yaml`, los dos *Dockerfile* y sus archivos de configuración, y crear el archivo de credenciales con `cp .env.example .env`, cambiando las contraseñas.
5. Construir las imágenes y levantar los servicios; los datos quedan en los volúmenes `wordpress_db_data` y `wordpress_wp_data`:

```
cd ~/unir/wordpress
podman-compose build        # construye localhost/wp-app y localhost/wp-nginx
podman-compose up -d        # crea la red, los volúmenes y los 3 contenedores
podman ps                   # wp-db debe aparecer como (healthy)
```

4. Despliegue y prueba

Una vez en marcha los contenedores, se abrió en el navegador del anfitrión la dirección `http://192.168.124.87:8080` y se completó el asistente de instalación de WordPress (idioma, título del sitio y usuario administrador). Después se realizaron las siguientes pruebas:

Tabla 3. Pruebas de funcionamiento.

Prueba	Cómo se comprueba	Resultado esperado
Servicio activo	`curl -I http://localhost:8080` dentro de la VM	HTTP 200 o 302 y cabecera `Server: nginx`
Acceso desde el anfitrión	Navegador en `http://192.168.124.87:8080`	Página de WordPress
Base de datos	Publicar una entrada de prueba	La entrada aparece en la portada
Subida de archivos	Subir una imagen de más de 2 MB a la biblioteca de medios	Se sube sin error (límite de 64 MB)
Enlaces permanentes	Ajustes → Enlaces permanentes → «Nombre de la entrada»	Las URL amigables abren la entrada
Seguridad básica	`curl -I http://localhost:8080/.htaccess`	HTTP 403 (archivos ocultos bloqueados)

Prueba	Cómo se comprueba	Resultado esperado
Persistencia	Reiniciar la VM con `sudo reboot`	Los contenedores arrancan solos y los datos se conservan

5. Problemas encontrados

- **Usuario de sistema sin rangos subordinados:** `useradd --system` no asigna rangos en `/etc/subuid` ni `/etc/subgid`, y Podman mostraba el aviso «Using rootless single mapping into the namespace. This might break some images». Se resolvió con `usermod --add-subuids/--add-subgids` y `podman system migrate`.
- **podman-compose con pipx:** al instalarlo en el usuario `podman` hubo que añadir `~/.local/bin` al PATH con `pipx ensurepath`.
- **Nombres cortos de imagen:** Podman no asume Docker Hub como registro predeterminado. Se usaron nombres completos, por ejemplo `docker.io/library/mariadb:11.4`.
- **Puertos privilegiados:** en modo sin privilegios Podman no puede abrir puertos inferiores a 1024, por lo que el 80 de Nginx se publica en el **8080**.
- **Límite de subida de PHP:** Nginx aceptaba 64 MB, pero PHP mantenía su límite de 2 MB. Se creó un *Dockerfile* que añade un `uploads.ini` con límites de 64 MB.
- **Arranque tras reinicio:** la política `restart` no basta en modo sin privilegios; fue necesario habilitar *linger* y el servicio `podman-restart` del usuario `podman`.

6. Vídeo de demostración de la máquina virtual

URL: pendiente de añadir. El vídeo muestra la creación de la máquina con virt-install y cloud-init, `podman ps`, el acceso a WordPress desde el anfitrión, la publicación de una entrada y el sitio funcionando tras reiniciar la VM.

Referencias

Amazon Web Services. (s. f.). *AWS Nitro System*. Recuperado el 5 de noviembre de 2026, de https://aws.amazon.com/ec2/nitro/

Canonical. (s. f.). *cloud-init documentation*. Recuperado el 5 de noviembre de 2026, de https://cloudinit.readthedocs.io/

Debian. (s. f.). *Debian Official Cloud Images*. Recuperado el 5 de noviembre de 2026, de https://cloud.debian.org/images/cloud/